The `update` method is most commonly used in controller actions where it merges data from a form into an existing database row:

```
def save_after_edit
  order = Order.find(params[:id])
  if order.update(order_params)
    redirect_to action: :index
  else
    render action: :edit
  end
end
```

We can combine the functions of reading a row and updating it using the class methods `update` and `update_all`. The `update` method takes an `id` parameter and a set of attributes. It fetches the corresponding row, updates the given attributes, saves the result to the database, and returns the model object.

```
order = Order.update(12, name: "Barney", email: "barney@bedrock.com")
```

We can pass `update` an array of IDs and an array of attribute value hashes, and it will update all the corresponding rows in the database, returning an array of model objects.

Finally, the `update_all` class method allows us to specify the `set` and `where` clauses of the SQL `update` statement. For example, the following increases the prices of all products with *Java* in their title by 10 percent:

```
result = Product.update_all("price = 1.1*price", "title like '%Java%'")
```

The return value of `update_all` depends on the database adapter; most (but not Oracle) return the number of rows that were changed in the database.

save, save!, create, and create!

It turns out that there are two versions of the `save` and `create` methods. The variants differ in the way they report errors.

- `save` returns `true` if the record was saved; it returns `nil` otherwise.
- `save!` returns `true` if the save succeeded; it raises an exception otherwise.